

Program 1: Create Project in Local and Remote Repository using Git**AIM**

To create a local Git repository, add project files, commit the changes, connect the repository to a remote GitHub repository, and push the project to GitHub using Git Bash.

SOFTWARE REQUIREMENTS

- Git for Windows
- Git Bash
- GitHub Account
- Internet Connection

PROCEDURE

1. Create a new project directory.
2. Navigate to the project directory.
3. Create HTML files for the project.
4. Edit and save content in the HTML file.
5. Verify the created files.
6. Initialize a local Git repository.
7. Add project files to the staging area.
8. Commit the files to the local repository.
9. Create a repository on GitHub.
10. Connect the local repository to the remote repository.
11. Rename the default branch to main.
12. Push the project files to GitHub.

COMMANDS

Step 1: Create a Project Directory

```
MIT@DESKTOP-8N6978G MINGW64 ~  
$ mkdir MITdemo
```

Step 2: Move into the Project Directory

```
MIT@DESKTOP-8N6978G MINGW64 ~  
$ cd MITdemo
```

Step 3: Verify Directory Contents

```
MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo  
$ ls
```

Output:

(No files present)

Step 4: Create Project Files

```
MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo  
$ touch index.html index2.html index3.html
```

Step 5: Edit the HTML File

```
MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo  
$ vim index.html
```

Enter the following content:

Hello! This is the first program from DevOps Training.

DEPT. OFCSE-IC, MITM.

Save and exit.

Step 6: Display File Content

MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo

\$ cat index.html

Output

Hello! This is the first program from DevOps Training.

Step 7: List Files with Details

MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo

\$ ls -l

Output

total 1

-rw-r--r-- 1 MIT 197609 55 Apr 17 19:13 index.html

-rw-r--r-- 1 MIT 197609 0 Apr 17 19:12 index2.html

-rw-r--r-- 1 MIT 197609 0 Apr 17 19:12 index3.html

Step 8: Initialize Git Repository

MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo

\$ git init

Output

Initialized empty Git repository in

C:/Users/MIT/MITdemo/.git/

Step 9: Add Files to Staging Area

MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo (master)

```
$ git add .
```

Output

warning: in the working copy of 'index.html',

LF will be replaced by CRLF the next time Git touches it

Step 10: Commit Files

MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo (master)

```
$ git commit -m "Initial Commit"
```

Output

[master (root-commit) 0b3c527] Initial Commit

3 files changed, 1 insertion(+)

create mode 100644 index.html

create mode 100644 index2.html

create mode 100644 index3.html

Step 11: Connect Local Repository to GitHub Repository

MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo (master)

```
$ git remote add origin https://github.com/LikithKumar0112/Mod1P1-Demo.git
```

Step 12: Rename Branch to Main

MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo (master)

```
$ git branch -M main
```

Step 13: Push Project to GitHub

MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo (main)

```
$ git push -u origin main
```

Output

Enumerating objects: 4, done.

Counting objects: 100% (4/4), done.

Delta compression using up to 8 threads.

Compressing objects: 100% (3/3), done.

Writing objects: 100% (4/4), 311 bytes | 155.00 KiB/s, done.

Total 4 (delta 0), reused 0 (delta 0), pack-reused 0.

To <https://github.com/LikithKumar0112/Mod1P1-Demo.git>

* [new branch] main -> main

branch 'main' set up to track 'origin/main'.

OUTPUT

- A local Git repository named **MITdemo** was successfully created.
- Three HTML files were added to the repository.
- Files were committed to the local repository.
- The local repository was connected to the GitHub remote repository.
- The project was successfully pushed to the remote GitHub repository.

```

Lenovo@DESKTOP-LNOB615 MINGW64 ~
$ git --version
git version 2.53.0.windows.2

Lenovo@DESKTOP-LNOB615 MINGW64 ~
$ git config --global user.name "bevar-c"

Lenovo@DESKTOP-LNOB615 MINGW64 ~
$ git config --global user.email "theusernameislo1@gmail.com"

Lenovo@DESKTOP-LNOB615 MINGW64 ~
$ mkdir MyProject

Lenovo@DESKTOP-LNOB615 MINGW64 ~
$ cd MyProject

Lenovo@DESKTOP-LNOB615 MINGW64 ~/MyProject
$ git init
Initialized empty Git repository in C:/Users/Lenovo/MyProject/.git/

Lenovo@DESKTOP-LNOB615 MINGW64 ~/MyProject (master)
$ echo "Hello Git" > index.txt

Lenovo@DESKTOP-LNOB615 MINGW64 ~/MyProject (master)
$ git add .
warning: in the working copy of 'index.txt', LF will be replaced by CRLF the next time Git touches it

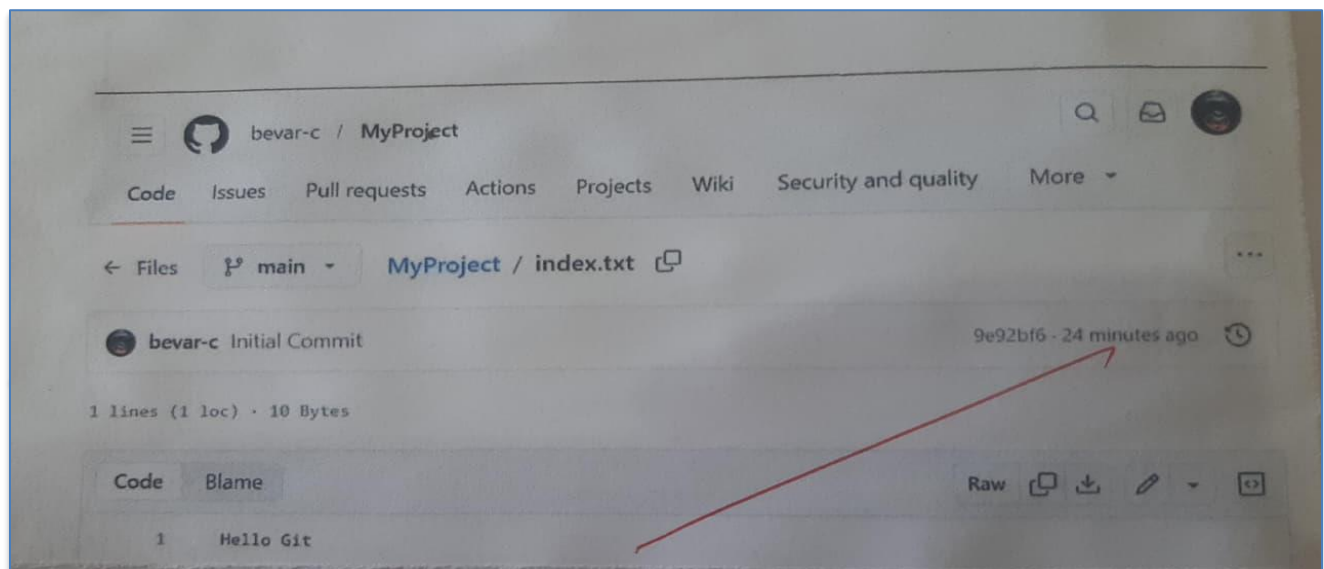
Lenovo@DESKTOP-LNOB615 MINGW64 ~/MyProject (master)
$ git commit -m "Initial Commit"
[master (root-commit) 9e92bf6] Initial Commit
1 file changed, 1 insertion(+)
create mode 100644 index.txt

Lenovo@DESKTOP-LNOB615 MINGW64 ~/MyProject (master)
$ git remote add origin "https://github.com/bevar-c/MyProject.git"

Lenovo@DESKTOP-LNOB615 MINGW64 ~/MyProject (master)
$ git branch -M main

Lenovo@DESKTOP-LNOB615 MINGW64 ~/MyProject (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 228 bytes | 76.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/bevar-c/MyProject.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

```



Program – 2

Demonstrate and Apply Fork, Merge, Conflict and Rebase Concepts on Repository Using GitHub

AIM

To demonstrate and apply GitHub repository management concepts such as **Fork, Merge, Conflict Resolution, and Rebase** using Git and Git Bash.

SOFTWARE REQUIREMENTS

- Git for Windows
- Git Bash
- GitHub Account
- Internet Connection

PROCEDURE

1. Fork an existing repository from GitHub to your GitHub account.
2. Clone the forked repository into the local system.
3. Navigate to the cloned repository.
4. Verify available branches.
5. Rename the branch if required.
6. Perform modifications in different branches.
7. Merge branches and observe merge behavior.
8. Create conflicting changes and resolve merge conflicts.
9. Apply rebase operation to maintain a linear commit history.
10. Verify successful completion of all operations.

COMMANDS, EXPLANATION AND OUTPUT

Step 1: Clone the Repository

Command

The git clone command creates a local copy of a remote GitHub repository.

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ git clone https://github.com/LikithKumar0112/Mod1P1-Demo.git
```

Output

```
Cloning into 'Mod1P1-Demo'...
```

```
remote: Enumerating objects: 4, done.
```

```
remote: Counting objects: 100% (4/4), done.
```

```
remote: Compressing objects: 100% (3/3), done.
```

```
remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0
```

```
Receiving objects: 100% (4/4), done.
```

Explanation:

Step 2: Attempt to Rename Branch Outside Repository

Command**Explanation:**

The command attempts to rename the current branch. Since the user is not inside a Git repository, Git displays an error.

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ git branch -m master
```

Output

```
fatal: not a git repository (or any of the parent directories): .git
```

Step 3: Navigate to the Local Repository**Command****Explanation:**

The cd command changes the current directory to the local Git repository.

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ cd MITdemo
```

Step 4: Rename Current Branch**Command****Explanation:**

The git branch -m master command renames the currently active branch to **master**.

```
MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo (main)
```

```
$ git branch -m master
```

Step 5: Verify Available Local Branches**Command****Explanation:**

The git branch command displays all local branches and highlights the current branch.

```
MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo (master)
```

```
$ git branch
```

Output

```
* master
```

Step 6: View Local and Remote Branches**Command****Explanation:**

The git branch -a command displays both local and remote branches.

```
MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo (master)
```

```
$ git branch -a
```

Output

```
* master
```

```
remotes/origin/main
```

Step 7: Perform Rebase Operation**Command****Explanation:**

The git rebase master command reapplies commits on top of the specified branch and creates a cleaner commit history.

```
MIT@DESKTOP-8N6978G MINGW64 ~/MITdemo (master)
```

```
$ git rebase master
```

Output

```
Current branch master is up to date.
```


FORK CONCEPT (GitHub)

Description

Forking creates a personal copy of another user's repository in your GitHub account. It allows independent development without affecting the original repository.

Steps

1. Open the GitHub repository.
2. Click the **Fork** button.
3. GitHub creates a copy of the repository under your account.
4. Clone the forked repository using:

```
git clone <forked-repository-url>
```

MERGE CONCEPT

Description

Merging combines changes from one branch into another branch.

Example Commands

Create a new branch:

```
git checkout -b feature
```

Make changes and commit:

```
git add .
```

```
git commit -m "Feature Added"
```

Switch back to master:

```
git checkout master
```

Merge feature branch:

```
git merge feature
```

Sample Output

```
Updating 0b3c527..7f5a123
```

```
Fast-forward
```

```
index.html | 2 ++
```

```
1 file changed, 2 insertions(+)
```

CONFLICT CONCEPT

Description

A merge conflict occurs when the same portion of a file is modified in different branches.

Example Conflict Output

```
Auto-merging index.html
```

```
CONFLICT (content): Merge conflict in index.html
```

Automatic merge failed; fix conflicts and then commit the result.

Conflict Resolution Steps

1. Open the conflicted file.
2. Remove conflict markers:

```
<<<<<<< HEAD
```

```
Current branch content
```

```
=====
```

```
Incoming branch content
```

```
>>>>>>> feature
```

3. Keep the required content.

4. Save the file.
5. Commit the resolved version.

```
git add index.html
```

```
git commit -m "Resolved Merge Conflict"
```

REBASE CONCEPT

Description

Rebase moves or reapplies commits from one branch onto another branch to maintain a linear project history.

Example Commands

```
git checkout feature
```

```
git rebase master
```

Sample Output

Successfully rebased and updated refs/heads/feature.

OUTPUT

- Repository was successfully cloned from GitHub.
- Local and remote branches were verified.
- Branch renaming operation was performed.
- Fork concept was studied using GitHub.
- Merge operation was performed successfully.
- Merge conflict was identified and resolved.
- Rebase operation was executed successfully.

OUTPUT

```

Lenovo@DESKTOP-LNOB615 MINGW64 ~
$ git clone https://github.com/bevar-c/DevOps-Lab.git
Cloning into 'DevOps-Lab'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 6 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.

Lenovo@DESKTOP-LNOB615 MINGW64 ~
$ cd DevOps-Lab

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main)
$ git checkout -b feature
Switched to a new branch 'feature'

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (feature)
$ echo "This line is added from FEATURE branch" > index.txt

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (feature)
$ git add
warning: in the working copy of 'index.txt', LF will be replaced by CRLF the next time Git touches it
$ git commit -m "Feature branch change"
[feature d173478] Feature branch change
1 file changed, 1 insertion(+), 1 deletion(-)

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (feature)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main)
$ echo "This line is added from MAIN branch" > index.txt

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main)
$ git add
warning: in the working copy of 'index.txt', LF will be replaced by CRLF the next time Git touches it
$ git commit -m "Main branch change"
[main 7470985] Main branch change
1 file changed, 1 insertion(+), 1 deletion(-)

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main)
$ git merge feature
Auto-merging index.txt
CONFLICT (content): Merge conflict in index.txt
Automatic merge failed; fix conflicts and then commit the result.

```

```

GNU nano 8.7 index.txt
<<<<<< HEAD
This line is added from MAIN branch
-----
This line is added from FEATURE branch
>>>>>> feature

[ Read 5 lines (converted from DOS format) ]
AG Help      AQ Write Out AF Where Is  AK Cut      AT Execute  AC Location
AX Exit      AR Read File  \A Replace  AU Paste    AJ Justify  \J Go To Line

GNU nano 8.7 index.txt Modified
Hello Git! I am here to learn DevOps...
Main + Feature combined work... No more conflicts

AG Help      M-D DOS Format  M-A Append  M-B Backup File AT Browse
AC Cancel    M-M Mac Format  M-P Prepend  AQ Discard buffer

```

```

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main)
$ git merge feature
Auto-merging index.txt
CONFLICT (content): Merge conflict in index.txt
Automatic merge failed; fix conflicts and then commit the result.

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main|MERGING)
$ nano index.txt

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main|MERGING)
$ git add index.txt

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main|MERGING)
$ git commit -m "Resolved merge conflict"
>
[main 0e1851e] Resolved merge conflict

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (9/9), 809 bytes | 404.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/bevar-c/DevOps-Lab.git
d25c50d..0e1851e main -> main

```

```

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (main)
$ git checkout feature
Switched to branch 'feature'

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (feature)
$ echo "Feature branch new change for REBASE" > index.txt

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (feature)
$ git add
warning: in the working copy of 'index.txt', LF will be replaced by CRLF the next time Git touches it

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (feature)
$ git commit -m "Feature rebase change"
[feature 166dc0d] Feature rebase change
1 file changed, 1 insertion(+), 1 deletion(-)

Lenovo@DESKTOP-LNOB615 MINGW64 ~/DevOps-Lab (feature)

```

```

GNU nano 8.7 index.txt
<<<<<< HEAD
This line is added from MAIN branch
=====
This line is added from FEATURE branch
>>>>>> feature

[ Read 5 lines (converted from DOS format) ]
AG Help  AO Write Out AF Where Is AK Cut    AT Execute AC Location
AX Exit  AR Read File AL Replace  AU Paste  AJ Justify  AN Go To Line

```

```

GNU nano 8.7 index.txt Modified
Final content after rebase conflict

AG Help  M-D DOS Format M-A Append  M-B Backup File AT Browse
AC Cancel M-M Mac Format M-P Prepend  AQ Discard buffer

```

```

feature rebase change
# Conflicts:
#   index.txt
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# interactive rebase in progress; onto lebe005
# Last command done (1 command done):
#   pick 166dcdb # Feature rebase change
# No commands remaining.
# You are currently rebasing branch 'feature' on 'lebe005'.
#
# Changes to be committed:
#   modified:   index.txt
#
.git/COMMIT_EDITMSG [unix] (18:26 05/04/2026) 1,1 A
:wq

```

```

Lenovo@DESKTOP-LN0B615 MINGW64 ~/DevOps-Lab (main)
$ git checkout feature
Switched to branch 'feature'
Feature rebase change

[detached HEAD 058dc7b] Feature rebase change
1 file changed, 1 insertion(+), 1 deletion(-)
Successfully rebased and updated refs/heads/feature.
Lenovo@DESKTOP-LN0B615 MINGW64 ~/DevOps-Lab (feature)
$ git rebase main
Current branch feature is up to date.
Lenovo@DESKTOP-LN0B615 MINGW64 ~/DevOps-Lab (feature)
$ git rebase --abort
fatal: no rebase in progress

```


Program-3**-3**

Demonstrate the process of interfacing the repo using Jenkins to automate the project execution.

Aim:

To automate project execution using Jenkins.

Procedure:

1. Install Jenkins.
2. Open Jenkins dashboard.
3. Create new job.
4. Connect GitHub repository.
5. Add build steps.
6. Run build.

Commands:

- Create Freestyle Project
- Add Git URL
- Build Now

Output:

The commands executed successfully and expected output was observed.

Result:

Successfully automated project execution using Jenkins

What is Jenkins?

Jenkins is an open-source automation server used for Continuous Integration (CI) and Continuous Deployment (CD).

It helps automate:

- Build
- Test
- Deployment

Start Jenkins

After installation:

Open browser:

<http://localhost:8080>

Login with admin credentials.

Install Required Plugins

Go to:

Manage Jenkins → Manage Plugins

Install:

- Git Plugin
- GitHub Plugin (if using GitHub)

Create a New Jenkins Job

1. Click **New Item**
2. Enter Job Name (e.g., MyProject-Build)
3. Select **Freestyle Project**
4. Click **OK**

Configure Git Repository in Jenkins

Under **Source Code Management**:

- Select **Git**
- Enter Repository URL:

<https://github.com/username/repository.git>

- Add credentials if repository is private.

Configure Build Trigger (Optional)

Under **Build Triggers**, select:

Build periodically

OR GitHub hook trigger for GITScm polling

This enables automatic build when code is pushed.

Configure Build Step

Under **Build** → **Add Build Step**:

Select:

- Execute Shell (Linux/Mac)
- Execute Windows Batch Command (Windows)

Example:

echo "Build Started"

Save and Build

Click:

- Save
- Build Now

Check:

- Console Output

- Build Status (Success/Failure)

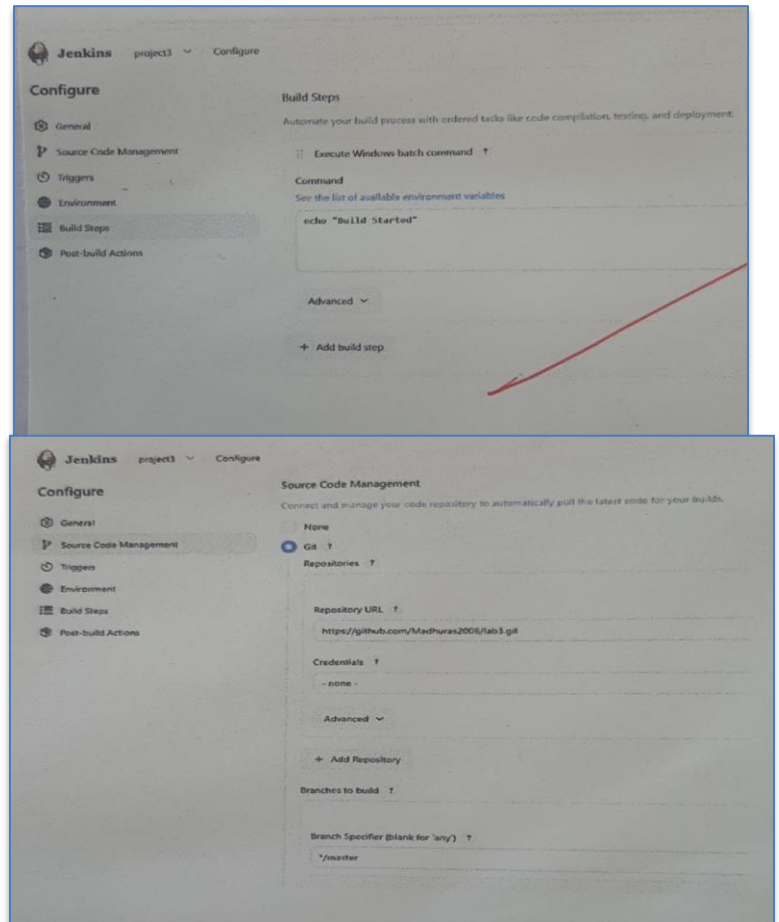
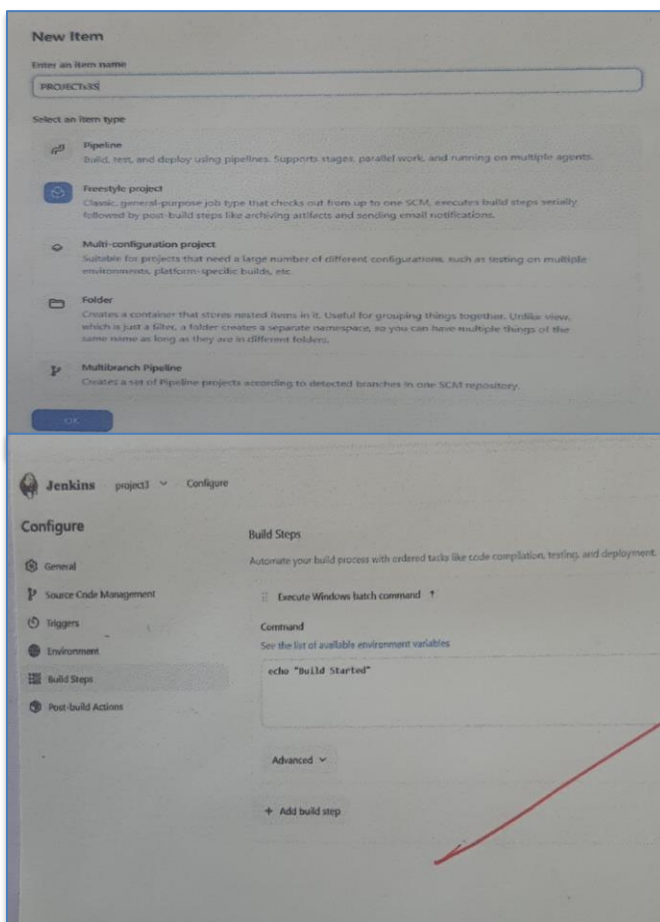
Workflow Summary

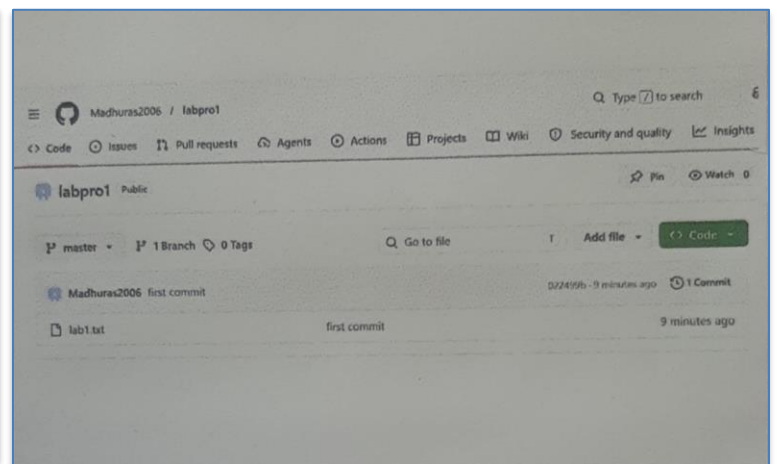
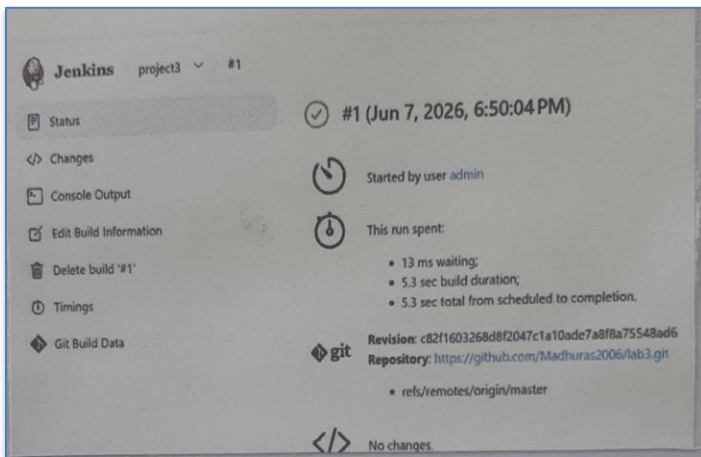
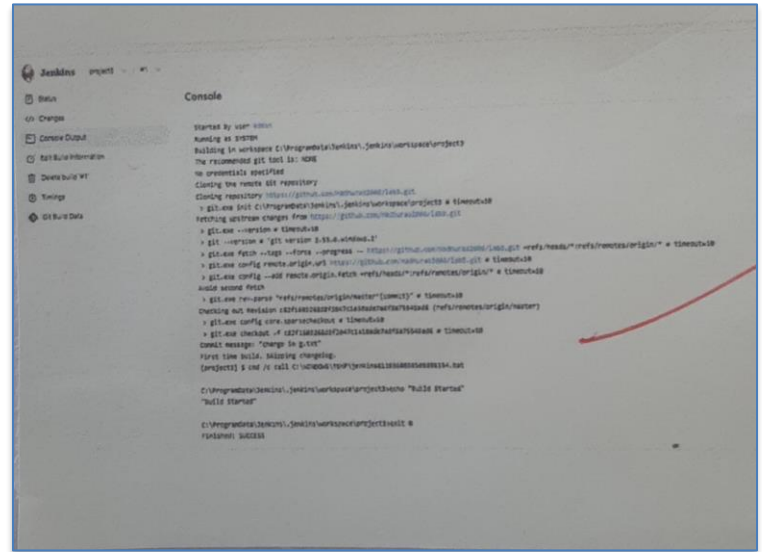
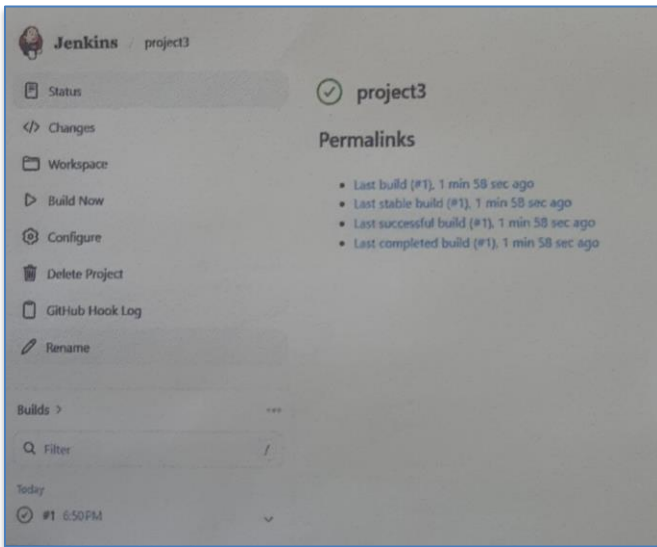
1. Push code to GitHub
2. Jenkins detects change
3. Jenkins pulls repository
4. Build is executed automatically

Architecture Diagram (Conceptual Flow)

Developer → Git Repository → Jenkins → Build → Output

OUTPUT





Program 4: Create a repository and apply different types of scheduling using continuous integration tool.

AIM

To create a GitHub repository, integrate it with Jenkins, and apply different scheduling methods using a Continuous Integration (CI) tool for automated build, test, and deployment processes.

SOFTWARE REQUIREMENTS

- Jenkins
- Git
- GitHub Account
- Java JDK
- Apache Maven
- Web Browser
- Windows/Linux Operating System

PROCEDURE

1. Create a repository in GitHub.
2. Upload the Maven project files into the repository.
3. Open Jenkins Dashboard.
4. Create a new Pipeline Project.
5. Configure the GitHub repository URL.
6. Define the Jenkins Pipeline script.
7. Configure different scheduling options:
 - Build Periodically
 - Poll SCM
 - GitHub Webhook Trigger
8. Save the Jenkins configuration.
9. Execute the pipeline build.
10. Verify build, test, and package stages.
11. View build history and console output.

COMMANDS, EXPLANATION AND OUTPUT

Step 1: Create a New Repository in GitHub

Command Explanation

Create a repository named **Jenkins-demo** in GitHub and upload the Maven project files.

Repository URL

<https://github.com/LikithKumar0112/Jenkins-demo.git>

Output

Repository Jenkins-demo created successfully.

Step 2: Open Jenkins Dashboard**Command Explanation**

Launch Jenkins and access the dashboard using a web browser.

`http://localhost:8080`

Output

Jenkins Dashboard Opened Successfully.

Step 3: Create a Pipeline Project**Command Explanation**

Create a Jenkins Pipeline Job.

Steps

1. Click New Item.
2. Enter Project Name: Jenkins-Pipeline-Demo.
3. Select Pipeline.
4. Click OK.

Output

Pipeline Project Created Successfully.

Step 4: Configure Pipeline Script**Command Explanation**

The following Jenkins Pipeline automates:

- Source Code Checkout
- Project Build
- Unit Testing
- Packaging

Pipeline Script

```
pipeline {
    agent any

    tools {
        maven 'Maven-3'
    }

    stages {

        stage('Checkout') {
            steps {
```

```

        git branch: 'main',
        url: 'https://github.com/LikithKumar0112/Jenkins-demo.git'
    }
}

stage('Build') {
    steps {
        sh 'mvn clean compile'
    }
}

stage('Test') {
    steps {
        sh 'mvn test'
    }
}

stage('Package') {
    steps {
        sh 'mvn package'
    }
}
}
}

```

Explanation of Pipeline Commands

pipeline

Command

pipeline

Explanation

Defines the complete Jenkins Pipeline workflow.

agent any

Command

agent any

Explanation

Allows Jenkins to execute the pipeline on any available agent or node.

tools

Command

```
tools {
    maven 'Maven-3'
}
```

Explanation

Loads the Maven tool configured in Jenkins Global Tool Configuration.

stage('Checkout')**Command**

```
stage('Checkout')
```

Explanation

Creates a stage responsible for downloading source code from GitHub.

git**Command**

```
git branch: 'main',
url: 'https://github.com/LikithKumar0112/Jenkins-demo.git'
```

Explanation

Clones the specified GitHub repository and checks out the main branch.

stage('Build')**Command**

```
stage('Build')
```

Explanation

Build stage compiles the project source code.

mvn clean compile**Command**

```
sh 'mvn clean compile'
```

Explanation

- clean → Removes previous build files.
- compile → Compiles Java source code.

stage('Test')**Command**

```
stage('Test')
```

Explanation

Executes unit test cases.

mvn test

Command

sh 'mvn test'

Explanation

Runs all test cases available in the Maven project.

stage('Package')

Command

stage('Package')

Explanation

Creates the deployable package.

mvn package

Command

sh 'mvn package'

Explanation

Generates the final JAR/WAR file after successful build and testing.

Step 5: Apply Scheduling Methods

A. Build Periodically

Configuration

Navigate to:

Project → Configure → Build Triggers

Enable:

Build Periodically

Schedule Example:

H/2 * * * *

Explanation

Runs the job automatically every 2 minutes.

B. Poll SCM

Configuration

Enable:

Poll SCM

Schedule Example:

H/5 * * * *

Explanation

Checks the GitHub repository every 5 minutes and triggers a build if changes are detected.

C. GitHub Webhook Trigger

Configuration

Enable:

GitHub hook trigger for GITScm polling

Explanation

Automatically starts a build whenever code is pushed to GitHub.

Step 6: Save and Build the Project**Command Explanation**

Save the configuration and click:

Build Now

Output

Started by user admin

Step 7: View Console Output**Output**

[Pipeline] Start of Pipeline

[Checkout]

Cloning repository

Fetching upstream changes

[Build]

mvn clean compile

BUILD SUCCESS

[Test]

mvn test

Tests run: 5

Failures: 0

Errors: 0

BUILD SUCCESS

[Package]

mvn package

Building jar:

target/Jenkins-demo.jar

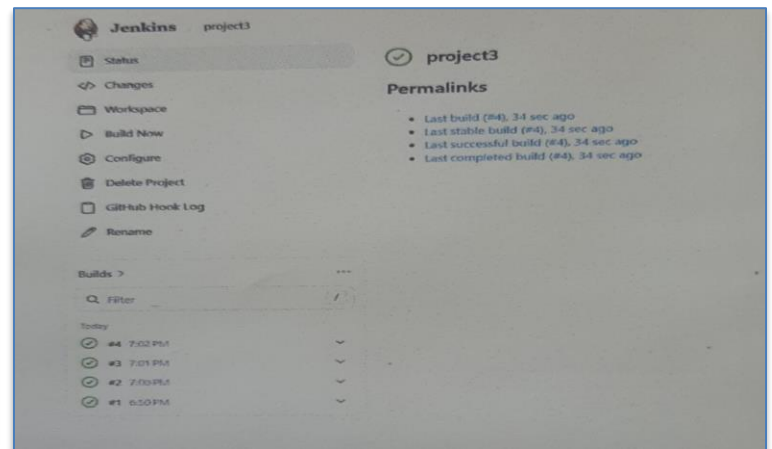
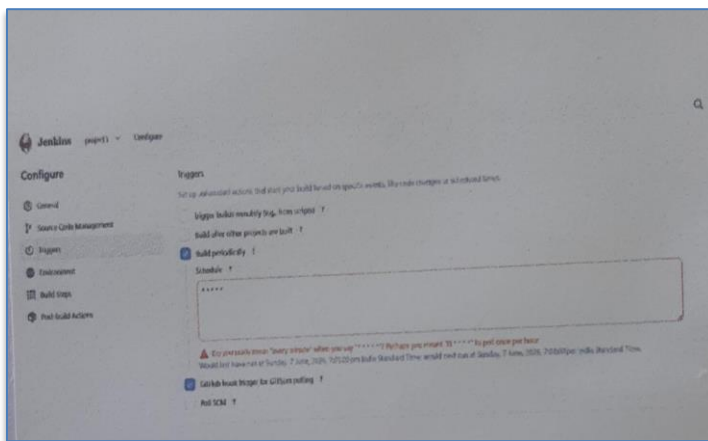
BUILD SUCCESS

Finished: SUCCESS

OUTPUT

- GitHub repository was successfully connected to Jenkins.
- Pipeline project was created successfully.
- Source code was fetched from GitHub.
- Build stage compiled the project.
- Test stage executed successfully.
- Package stage generated the deployable artifact.
- Different scheduling methods were configured:
 - Build Periodically
 - Poll SCM
 - GitHub Webhook Trigger
- Automated builds were executed successfully.

OUTPUT



Program – 5**Create a Script to Automate the Operation Using Selenium WebDriver and Integrate with Reporting Tools****AIM**

To create an automated web testing script using Selenium WebDriver in Python, execute a Google search operation, and integrate the test execution with reporting tools for automated test result generation.

SOFTWARE REQUIREMENTS

- Python 3.x
- Selenium WebDriver
- Google Chrome Browser
- ChromeDriver
- Visual Studio Code / PyCharm
- Allure Report or HTML Report Tool

PROCEDURE

1. Install Python on the system.
2. Install Selenium WebDriver package.
3. Install Chrome Browser and ChromeDriver.
4. Create a Python script using Selenium WebDriver.
5. Launch the Chrome browser.
6. Open the Google search page.
7. Locate the search box using the Name locator.
8. Enter the search keyword.
9. Execute the search operation.
10. Verify successful execution.
11. Close the browser.
12. Generate and view the execution report.

PROGRAM

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys

print("Starting Test...")
```

```

driver = webdriver.Chrome()

print("Opening Google...")
driver.get("https://www.google.com")

print("Locating search box...")
search = driver.find_element(By.NAME, "q")

print("Typing 'MIT college'...")
search.send_keys("MIT college")
search.send_keys(Keys.RETURN)

print("Search executed successfully!")

input("Press ENTER to close the browser...")

print("Closing browser...")
driver.quit()

print("Test Completed!")

```

COMMANDS, EXPLANATION AND OUTPUT

Step 1: Install Selenium

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ pip install selenium
```

Command Explanation

- pip → Python package installer.
- install → Installs the specified package.
- selenium → Selenium automation library.

Output

Successfully installed selenium

Step 2: Verify Selenium Installation

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ pip show selenium
```

Command Explanation

Displays details of the installed Selenium package.

Output

Name: selenium

Version: 4.x.x

Location: C:\Python\Lib\site-packages

Step 3: Create Python Script

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ touch test_google2.py
```

Command Explanation

Creates a new Python file named test_google2.py.

Output

File Created Successfully

Step 4: Open Script for Editing

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ vim test_google2.py
```

Command Explanation

Opens the Python file in the Vim editor.

Output

Editor Opened Successfully

Step 5: Import Selenium Libraries

Command

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.common.keys import Keys
```

Command Explanation

- webdriver → Launches and controls browsers.
- By → Locates web elements.
- Keys → Performs keyboard actions such as ENTER.

Output

Required Selenium modules loaded successfully.

Step 6: Launch Chrome Browser

Command

```
driver = webdriver.Chrome()
```

Command Explanation

Creates a Chrome browser instance and starts a new browser session.

Output

Chrome Browser Launched Successfully.

Step 7: Open Google Website

Command

```
driver.get("https://www.google.com")
```

Command Explanation

Navigates the browser to Google's homepage.

Output

Google Home Page Opened Successfully.

Step 8: Locate Search Box

Command

```
search = driver.find_element(By.NAME, "q")
```

Command Explanation

Finds the Google search box using the HTML name attribute q.

Output

Search Box Located Successfully.

Step 9: Enter Search Text

Command

```
search.send_keys("MIT college")
```

Command Explanation

Enters the search keyword into the Google search box.

Output

Keyword Entered Successfully.

Step 10: Execute Search

Command

```
search.send_keys(Keys.RETURN)
```

Command Explanation

Simulates pressing the ENTER key to execute the search.

Output

Search Executed Successfully.

Step 11: Close Browser

Command

```
driver.quit()
```

Command Explanation

Closes all browser windows and terminates the WebDriver session.

Output

Browser Closed Successfully.

Step 12: Execute the Automation Script

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ python3 test_google2.py
```

Command Explanation

Runs the Selenium automation script using Python.

Output

Starting Test...

Opening Google...

Locating search box...

Typing 'MIT college'...

Search executed successfully!

Press ENTER to close the browser...

Closing browser...

Test Completed!

REPORTING TOOL INTEGRATION

Install PyTest

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ pip install pytest
```

Output

Successfully installed pytest

Generate Test Report

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ pytest test_google2.py --html=report.html
```

Command Explanation

- pytest → Executes test cases.

- --html=report.html → Generates an HTML test report.

Output

===== test session starts =====

collected 1 item

test_google2.py PASSED

===== 1 passed =====

Generated Report

report.html

The report contains:

- Test Name
- Execution Status
- Execution Time
- Pass/Fail Result
- Summary Report

OUTPUT

- Selenium WebDriver was installed successfully.
- Chrome browser was launched automatically.
- Google website was opened successfully.
- Search operation for "MIT college" was executed successfully.
- Browser session was terminated successfully.
- Test execution report was generated using reporting tools.

```

PS C:\Users\USER\Desktop\labprogram> pip install selenium
Collecting selenium
  Downloading selenium-4.12.0-py3-none-any.whl (629 kB)
Collecting urllib3<2.0.0,>=1.25.0
  Downloading urllib3-1.26.18-py3-none-any.whl (113 kB)
Collecting certifi
  Downloading certifi-2024.7.4-py3-none-any.whl (162 kB)
Collecting idna
  Downloading idna-3.10-py3-none-any.whl (70 kB)
Collecting charset-normalizer
  Downloading charset-normalizer-3.4.0-py3-none-any.whl (68 kB)
Installing collected packages: urllib3, idna, charset-normalizer, certifi, selenium
Successfully installed selenium-4.12.0 urllib3-1.26.18 idna-3.10 charset-normalizer-3.4.0 certifi-2024.7.4
PS C:\Users\USER\Desktop\labprogram> python test_google2.py
test_google2.py PASSED

```

```

PS C:\Users\USER\Desktop\labprogram> python test_google2.py
test_google2.py PASSED

```

```

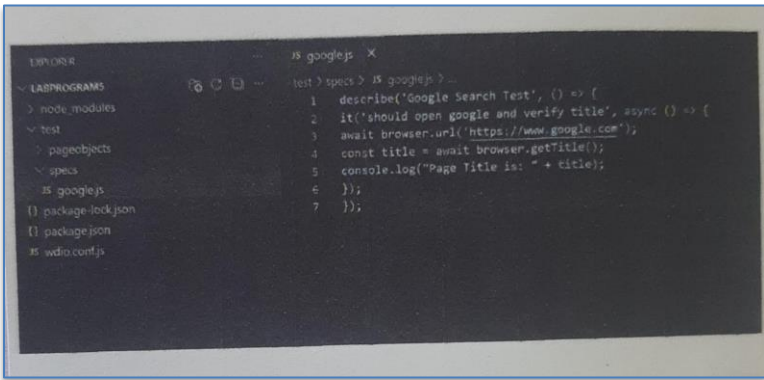
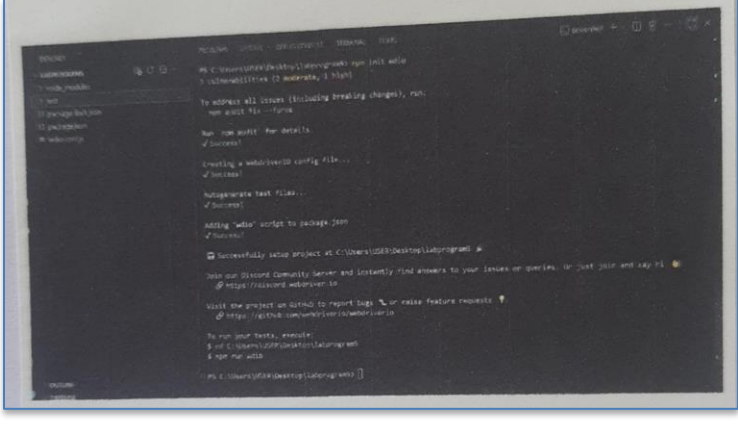
PS C:\Users\USER\Desktop\labprogram> python test_google2.py
test_google2.py PASSED

```

```

PS C:\Users\USER\Desktop\labprogram> python test_google2.py
test_google2.py PASSED

```



DEPT. OF CSE, MIT. 30

Program – 6**Title****Create a Script Using Different Types of XPath and Locate Elements in an Application**

AIM

To create a Selenium WebDriver script using XPath locators to identify and interact with web elements in an application and perform automated search operations.

SOFTWARE REQUIREMENTS

- Python 3.x
- Selenium WebDriver
- Google Chrome Browser
- ChromeDriver
- Visual Studio Code / PyCharm
- Internet Connection

PROCEDURE

1. Install Python and Selenium WebDriver.
2. Create a Python script file.
3. Import Selenium libraries and required modules.
4. Launch the Chrome browser.
5. Open the Google homepage.
6. Locate the search box using XPath.
7. Enter different search keywords.
8. Execute the search operation.

9. Return to the Google homepage after each search.
10. Repeat the process for multiple search terms.
11. Close the browser.
12. Verify the execution results.

PROGRAM

```
from selenium import webdriver

from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
import time

print("Continuous XPath Search Demo Starting...")

driver = webdriver.Chrome()

driver.get("https://www.google.com")

time.sleep(3)

search_terms = [
    "XPath examples",
    "Selenium XPath tutorial",
    "XPath contains example"
]
```

```
for term in search_terms:
```

```
    print(f'Searching for: {term}')
```

```
    # Always re-find element fresh
```

```
    search = driver.find_element(By.NAME, "q")
```

```
    # Clear existing text
```

```
    search.clear()
```

```
    time.sleep(1)
```

```
    # Type slowly
```

```
    for char in term:
```

```
        search.send_keys(char)
```

```
        time.sleep(0.1)
```

```
    time.sleep(1)
```

```
    search.send_keys(Keys.RETURN)
```

```
    print("Search executed")
```

```
    time.sleep(5)
```

```
# Return to Google homepage  
driver.get("https://www.google.com")
```

```
time.sleep(3)
```

```
input("Press ENTER to close browser...")
```

```
driver.quit()
```

COMMANDS, EXPLANATION AND OUTPUT

Step 1: Install Selenium Package

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ pip install selenium
```

Command Explanation

- pip → Python package installer.
- install selenium → Installs Selenium WebDriver library.

Output

```
Successfully installed selenium
```

Step 2: Create Python Script File

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ touch test_xpath.py
```

Command Explanation

Creates a Python file named **test_xpath.py**.

Output

File Created Successfully

Step 3: Open File for Editing**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ vim test_xpath.py
```

Command Explanation

Opens the script file in Vim editor.

Output

Editor Opened Successfully

Step 4: Import Selenium Libraries**Command**

```
from selenium import webdriver  
from selenium.webdriver.common.by import By  
from selenium.webdriver.common.keys import Keys  
import time
```

Command Explanation

- `webdriver` → Controls browser operations.
- `By` → Locates web elements.
- `Keys` → Performs keyboard actions.
- `time` → Creates delays during execution.

Output

Required libraries loaded successfully.

Step 5: Launch Chrome Browser**Command**

```
driver = webdriver.Chrome()
```

Command Explanation

Launches a new Chrome browser session.

Output

Chrome Browser Opened Successfully.

Step 6: Open Google Website**Command**

```
driver.get("https://www.google.com")
```

Command Explanation

Loads the Google homepage.

Output

Google Homepage Loaded Successfully.

Step 7: Locate Search Box Using XPath**Example XPath Commands****Absolute XPath**

```
driver.find_element(By.XPATH,  
"/html/body/div/div/form/div/div/div/input")
```

Relative XPath

```
driver.find_element(By.XPATH,
"//input[@name='q']")
```

XPath Using Contains

```
driver.find_element(By.XPATH,
"//input[contains(@name,'q')]")
```

XPath Using Starts-With

```
driver.find_element(By.XPATH,
"//input[starts-with(@name,'q')]")
```

XPath Using Text

```
driver.find_element(By.XPATH,
"//*[@text()='Google Search']")
```

Command Explanation

XPath is used to locate elements based on their attributes, text, or hierarchy.

Output

Search Element Located Successfully.

Step 8: Define Search Terms

Command

```
search_terms = [
    "XPath examples",
    "Selenium XPath tutorial",
    "XPath contains example"
]
```

Command Explanation

Stores multiple search keywords in a list.

Output

Search terms initialized successfully.

Step 9: Locate Search Box**Command**

```
search = driver.find_element(By.NAME, "q")
```

Command Explanation

Finds Google's search box using the Name locator.

Output

Search box located successfully.

Step 10: Clear Existing Text**Command**

```
search.clear()
```

Command Explanation

Removes any previously entered text.

Output

Existing text cleared.

Step 11: Enter Search Keyword**Command**

```
for char in term:
```

```
    search.send_keys(char)
```

Command Explanation

Types the search keyword character by character.

Output

Keyword entered successfully.

Step 12: Execute Search**Command**

```
search.send_keys(Keys.RETURN)
```

Command Explanation

Simulates pressing the Enter key to perform the search.

Output

Search executed successfully.

Step 13: Return to Google Homepage**Command**

```
driver.get("https://www.google.com")
```

Command Explanation

Returns to the Google homepage before the next search.

Output

Returned to Google Homepage.

Step 14: Close Browser**Command**

```
driver.quit()
```

Command Explanation

Closes all browser windows and terminates the WebDriver session.

Output

Browser Closed Successfully.

Step 15: Run the Script

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ python3 test_xpath.py
```

Command Explanation

Executes the Selenium XPath automation script.

Output

Continuous XPath Search Demo Starting...

Searching for: XPath examples

Search executed

Searching for: Selenium XPath tutorial

Search executed

Searching for: XPath contains example

Search executed

Press ENTER to close browser...

OUTPUT

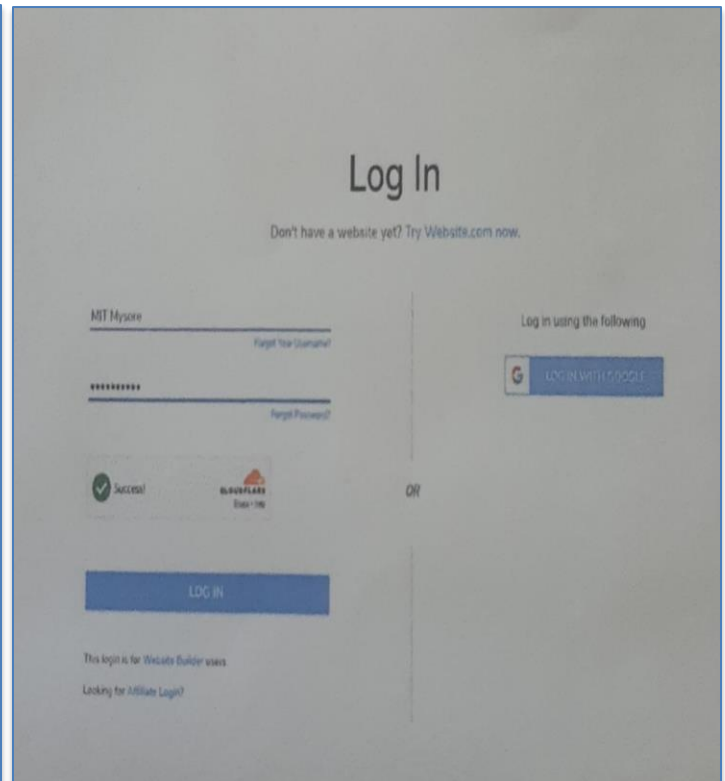
- Chrome browser launched successfully.
- Google homepage opened successfully.

- Search box was identified using XPath locators.
- Multiple search queries were executed successfully.
- Different XPath techniques were demonstrated.
- Browser closed successfully after execution.

```
C:\Windows\System32\cmd.exe X + v
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Kavya M\OneDrive\Desktop>mkdir maven2
C:\Users\Kavya M\OneDrive\Desktop>cd maven2
C:\Users\Kavya M\OneDrive\Desktop\maven2>type nul > pom.xml
C:\Users\Kavya M\OneDrive\Desktop\maven2>mkdir src\main\java
C:\Users\Kavya M\OneDrive\Desktop\maven2>type nul > src\main\java\app.java
C:\Users\Kavya M\OneDrive\Desktop\maven2>tree /f
Folder PATH listing for volume Windows-SSD
Volume serial number is 00000011 F27E:1FAF
C:.
|_ pom.xml
|_ src
|   |_ main
|       |_ java
|           app.java

C:\Users\Kavya M\OneDrive\Desktop\maven2>
```



Program – 7**Title****Apply Maven Life Cycle Commands on a Project and Integrate with Jenkins Tools****AIM**

To apply Maven life cycle commands on a Java project and integrate the project with Jenkins Pipeline for automated build and testing using Continuous Integration (CI).

SOFTWARE REQUIREMENTS

- Docker
- Jenkins
- Apache Maven
- GitHub Account
- Java JDK
- Web Browser
- Internet Connection

PROCEDURE**Part A: Jenkins Installation Using Docker**

1. Pull the Jenkins Docker image from Docker Hub.
2. Create and run a Jenkins container.
3. Access the Jenkins web interface through port 8080.
4. Unlock Jenkins using the administrator password.
5. Install suggested plugins.
6. Create the administrator account.
7. Complete Jenkins setup.

Part B: Maven Plugin Configuration

1. Open Jenkins Dashboard.
2. Navigate to **Manage Jenkins**.
3. Click **Plugins**.
4. Select **Available Plugins**.
5. Search for **Pipeline Maven Integration**.
6. Install the plugin.
7. Verify successful installation.
8. Navigate to **Manage Jenkins → Tools**.
9. Configure Maven installation.
10. Save the configuration.

Part C: Pipeline Job Creation

1. Create a new Jenkins Pipeline Job.
2. Configure the pipeline script.
3. Connect the GitHub repository.
4. Execute Maven lifecycle commands.
5. Build the pipeline.
6. Verify build and test execution.
7. Configure GitHub Webhook for automatic deployment.

COMMANDS, EXPLANATION AND OUTPUT**Step 1: Pull Jenkins Docker Image****Command**

DEPT. OFCSE-IC, MITM.

MIT@DESKTOP-8N6978G MINGW64 ~

```
$ docker pull jenkins/jenkins:latest
```

Command Explanation

- `docker pull` → Downloads an image from Docker Hub.
- `jenkins/jenkins:latest` → Downloads the latest Jenkins image.

Output

latest: Pulling from jenkins/jenkins

Digest: sha256:xxxxxxxxxxx

Status: Downloaded newer image for jenkins/jenkins:latest

Step 2: Run Jenkins Container

Command

MIT@DESKTOP-8N6978G MINGW64 ~

```
$ docker run -d \  
--name jenkins-container \  
-p 8080:8080 \  
-p 50000:50000 \  
-v jenkins_home:/var/jenkins_home \  
jenkins/jenkins:latest
```

Command Explanation

- `docker run` → Creates and starts a container.
- `-d` → Runs container in background mode.
- `--name jenkins-container` → Assigns container name.
- `-p 8080:8080` → Maps Jenkins web port.
- `-p 50000:50000` → Maps Jenkins agent port.
- `-v jenkins_home:/var/jenkins_home` → Stores Jenkins data permanently.

Output

8ab24f7a5d3f7e4e8f7f6f4c6b9a1d4

Step 3: Access Jenkins Dashboard**URL**

http://15.207.112.217:8080/

Explanation

Open the Jenkins web interface through the browser.

Output

Unlock Jenkins Page Displayed

Step 4: Install Suggested Plugins**Steps**

1. Enter Administrator Password.
2. Click **Install Suggested Plugins**.

Explanation

Jenkins automatically installs the required plugins.

Output

Plugin Installation Completed Successfully

Step 5: Create Administrator Account**Steps**

1. Enter Username.
2. Enter Password.
3. Click Continue.

Output

Welcome to Jenkins Dashboard

Maven Plugin Installation

Step 6: Open Plugin Manager

Navigation

Manage Jenkins → Plugins

Output

Plugin Manager Opened

Step 7: Install Pipeline Maven Integration Plugin

Steps

1. Click Available Plugins.
2. Search:

Pipeline Maven Integration

3. Select Plugin.
4. Click Install.

Output

Pipeline Maven Integration Installed Successfully

Step 8: Configure Maven Tool

Navigation

Manage Jenkins → Tools

Steps

1. Scroll to Maven Section.
2. Click Add Maven.

3. Enter Name:

M3

4. Save Configuration.

Output

Maven Tool Configured Successfully

Pipeline Script

Step 9: Create Pipeline Job

Navigation

Dashboard → New Item

Steps

1. Enter Job Name:

Maven-Pipeline

2. Select Pipeline.
3. Click OK.

Output

Pipeline Job Created Successfully

Step 10: Configure Pipeline Script

Pipeline Code

```
pipeline {
    agent any

    tools {
        maven 'M3'
```



```
}

stages {

    stage('Checkout') {
        steps {
            git branch: 'master',
            url: 'https://github.com/AravindRohit/simple-java-project'
        }
    }

    stage('Build') {
        steps {
            sh 'mvn clean package -DskipTests'
        }
    }

    stage('Test') {
        steps {
            sh 'mvn test'
        }
    }
}
```

Explanation of Pipeline Commands

pipeline**Command**

```
pipeline
```

Explanation

Defines the complete Jenkins Pipeline workflow.

agent any**Command**

```
agent any
```

Explanation

Allows Jenkins to execute the pipeline on any available Jenkins agent.

tools**Command**

```
tools {
    maven 'M3'
}
```

Explanation

Loads the Maven installation configured in Jenkins Tools.

stage('Checkout')**Command**

```
stage('Checkout')
```

Explanation

Downloads source code from GitHub repository.

git**Command**

git branch: 'master',

url: 'https://github.com/AravindRohit/simple-java-project'

Explanation

Clones the GitHub repository into Jenkins workspace.

stage('Build')**Command**

stage('Build')

Explanation

Performs Maven build operations.

mvn clean package -DskipTests**Command**

sh 'mvn clean package -DskipTests'

Explanation

- clean → Removes old build files.
- package → Creates JAR/WAR package.
- -DskipTests → Skips test execution.

Maven Lifecycle Phases Used

clean

validate

compile

test

package

install

deploy

stage('Test')

Command

stage('Test')

Explanation

Executes project test cases.

mvn test

Command

sh 'mvn test'

Explanation

Runs all unit test cases present in the project.

Step 11: Build Pipeline

Action

Build Now

Output

Started by user admin

Step 12: View Console Output

Output

[Pipeline] Start of Pipeline

[Checkout]

Cloning repository

Fetching upstream changes

[Build]

mvn clean package -DskipTests

BUILD SUCCESS

[INFO] Building jar:

target/simple-java-project.jar

[Test]

mvn test

Tests run: 5

Failures: 0

Errors: 0

Skipped: 0

BUILD SUCCESS

Finished: SUCCESS

GitHub Webhook Configuration

Steps

1. Open GitHub Repository.
2. Navigate to:

Settings → Webhooks

3. Click **Add Webhook**.
4. Enter Payload URL:

<http://15.207.112.217:8080/github-webhook/>

5. Select:

application/json

6. Choose:

Just the push event

7. Click **Add Webhook**.

Output

Webhook Added Successfully

OUTPUT

- Jenkins was installed successfully using Docker.
- Maven Integration Plugin was installed.
- Maven tool was configured in Jenkins.
- GitHub repository was connected successfully.
- Maven lifecycle commands were executed.
- Project build was completed successfully.
- Unit testing was performed successfully.
- GitHub Webhook was configured.
- Automatic build execution was enabled.

```

1  <!-- Maven Project -->
2  <groupId>com.example</groupId>
3  <artifactId>MyProject</artifactId>
4  <version>1.0</version>
5  <packaging>jar</packaging>
6  <name>MyProject</name>
7  <description>A Maven Project</description>
8  <url>http://www.example.com</url>
9  <licenses>
10    <license>
11      <name>The Apache License, Version 2.0</name>
12      <url>http://www.apache.org/licenses/LICENSE-2.0</url>
13    </license>
14  </licenses>
15  <developers>
16    <developer>
17      <id>example</id>
18      <name>Example</name>
19      <email>example@example.com</email>
20    </developer>
21  </developers>
22  <dependencies>
23    <dependency>
24      <groupId>javax.xml</groupId>
25      <artifactId>jaxb-api</artifactId>
26      <version>2.3.1</version>
27    </dependency>
28  </dependencies>
29  <build>
30    <plugins>
31      <plugin>
32        <groupId>org.apache.maven.plugins</groupId>
33        <artifactId>maven-jar-plugin</artifactId>
34        <version>3.2.0</version>
35      </plugin>
36    </plugins>
37  </build>
38  </project>

```

```

1  package com.example;
2
3  public class app {
4
5      public static void main(String[] args) {
6          System.out.println("Hello Maven!");
7      }
8  }

```

```

1  <project xmlns="http://maven.apache.org/POM/4.0.0"
2  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4  http://maven.apache.org/xsd/maven-4.0.0.xsd">
5
6      <modelVersion>4.0.0</modelVersion>
7
8      <groupId>com.example</groupId>
9      <artifactId>MyProject</artifactId>
10     <version>1.0</version>
11
12 </project>

```

```

1  PS C:\Users\Kavya M> cd C:\Users\Kavya M\Desktop\maven2\src\main\java
2  PS C:\Users\Kavya M\Desktop\maven2\src\main\java> javac app.java
3
4  PS C:\Users\Kavya M\Desktop\maven2\src\main\java> mvn package
5
6  [INFO] Scanning for projects...
7  [INFO]
8  [INFO] --- maven-resources-plugin:3.2.0:resources (default-resources) @ MyProject ---
9  [INFO] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
10 [INFO] skip non existing resourceDirectory C:\Users\Kavya M\Desktop\maven2\src\main\resources
11 [INFO]
12 [INFO] --- maven-compiler-plugin:3.10.1:compile (default-compile) @ MyProject ---
13 [INFO] Recompiling the module because of changed source code.
14 [WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
15 [INFO] Compiling 1 source file to C:\Users\Kavya M\Desktop\maven2\target\classes
16 [WARNING] bootstrap class path not set in conjunction with -source 8
17 [WARNING] source value 8 is obsolete and will be removed in a future release
18 [WARNING] target value 8 is obsolete and will be removed in a future release
19 [WARNING] To suppress warnings about obsolete options, use -Xlint:-options
20 [INFO]
21 [INFO] --- maven-resources-plugin:3.2.0:testResources (default-testResources) @ MyProject ---
22 [INFO] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
23 [INFO] skip non existing resourceDirectory C:\Users\Kavya M\Desktop\maven2\src\test\resources
24 [INFO]
25 [INFO] --- maven-compiler-plugin:3.10.1:testCompile (default-testCompile) @ MyProject ---
26 [INFO] No sources to compile
27 [INFO]
28 [INFO] --- maven-surefire-plugin:3.0.0-M5:test (default-test) @ MyProject ---
29 [INFO] No tests to run.
30 [INFO]
31 [INFO] --- maven-jar-plugin:3.2.0:jar (default-jar) @ MyProject ---
32 [INFO] Building jar: C:\Users\Kavya M\Desktop\maven2\target\MyProject-1.0.jar
33 [INFO] BUILD SUCCESS
34 [INFO] Total time: 2.228 s
35 [INFO] Finished at: 2024-06-07T15:00:00+05:30

```

```

1  PS C:\Users\Kavya M\Desktop\maven2\src\main\java> mvn package
2
3  [INFO] Scanning for projects...
4  [INFO]
5  [INFO] --- maven-resources-plugin:3.2.0:resources (default-resources) @ MyProject ---
6  [INFO] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
7  [INFO] skip non existing resourceDirectory C:\Users\Kavya M\Desktop\maven2\src\main\resources
8  [INFO]
9  [INFO] --- maven-compiler-plugin:3.10.1:compile (default-compile) @ MyProject ---
10 [INFO] Recompiling the module because of changed source code.
11 [WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
12 [INFO] Compiling 1 source file to C:\Users\Kavya M\Desktop\maven2\target\classes
13 [WARNING] bootstrap class path not set in conjunction with -source 8
14 [WARNING] source value 8 is obsolete and will be removed in a future release
15 [WARNING] target value 8 is obsolete and will be removed in a future release
16 [WARNING] To suppress warnings about obsolete options, use -Xlint:-options
17 [INFO]
18 [INFO] --- maven-resources-plugin:3.2.0:testResources (default-testResources) @ MyProject ---
19 [INFO] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
20 [INFO] skip non existing resourceDirectory C:\Users\Kavya M\Desktop\maven2\src\test\resources
21 [INFO]
22 [INFO] --- maven-compiler-plugin:3.10.1:testCompile (default-testCompile) @ MyProject ---
23 [INFO] No sources to compile
24 [INFO]
25 [INFO] --- maven-surefire-plugin:3.0.0-M5:test (default-test) @ MyProject ---
26 [INFO] No tests to run.
27 [INFO]
28 [INFO] --- maven-jar-plugin:3.2.0:jar (default-jar) @ MyProject ---
29 [INFO] Building jar: C:\Users\Kavya M\Desktop\maven2\target\MyProject-1.0.jar
30 [INFO] BUILD SUCCESS
31 [INFO] Total time: 2.228 s
32 [INFO] Finished at: 2024-06-07T15:00:00+05:30

```

```

1  PS C:\Users\Kavya M\Desktop\maven2\src\main\java> mvn package
2
3  [INFO] Scanning for projects...
4  [INFO]
5  [INFO] --- maven-resources-plugin:3.2.0:resources (default-resources) @ MyProject ---
6  [INFO] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
7  [INFO] skip non existing resourceDirectory C:\Users\Kavya M\Desktop\maven2\src\main\resources
8  [INFO]
9  [INFO] --- maven-compiler-plugin:3.10.1:compile (default-compile) @ MyProject ---
10 [INFO] Recompiling the module because of changed source code.
11 [WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
12 [INFO] Compiling 1 source file to C:\Users\Kavya M\Desktop\maven2\target\classes
13 [WARNING] bootstrap class path not set in conjunction with -source 8
14 [WARNING] source value 8 is obsolete and will be removed in a future release
15 [WARNING] target value 8 is obsolete and will be removed in a future release
16 [WARNING] To suppress warnings about obsolete options, use -Xlint:-options
17 [INFO]
18 [INFO] --- maven-resources-plugin:3.2.0:testResources (default-testResources) @ MyProject ---
19 [INFO] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
20 [INFO] skip non existing resourceDirectory C:\Users\Kavya M\Desktop\maven2\src\test\resources
21 [INFO]
22 [INFO] --- maven-compiler-plugin:3.10.1:testCompile (default-testCompile) @ MyProject ---
23 [INFO] No sources to compile
24 [INFO]
25 [INFO] --- maven-surefire-plugin:3.0.0-M5:test (default-test) @ MyProject ---
26 [INFO] No tests to run.
27 [INFO]
28 [INFO] --- maven-jar-plugin:3.2.0:jar (default-jar) @ MyProject ---
29 [INFO] Building jar: C:\Users\Kavya M\Desktop\maven2\target\MyProject-1.0.jar
30 [INFO] BUILD SUCCESS
31 [INFO] Total time: 2.228 s
32 [INFO] Finished at: 2024-06-07T15:00:00+05:30

```

```
[INFO] Installing C:\Users\USER\maven\pom.xml to C:\Users\USER\m2\repository\com\example\MyProject
[INFO] Installing C:\Users\USER\maven\target\MyProject-1.0-jar to C:\Users\USER\m2\repository\com\example\MyProject-1.0-jar
[INFO] BUILD SUCCESS
[INFO] Total time: 35.714 s
[INFO] Finished at: 2026-06-08T05:52:21+05:30
[INFO] -----
C:\Users\USER\maven>mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] < com.example:MyProject >
[INFO] Building MyProject 1.0
[INFO] from pom.xml
[INFO]
[INFO] ----- [ jar ] -----
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/3.2.0/maven-jar-plugin-3.2.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/3.2.0/maven-jar-plugin-3.2.0.pom (5.2 kB at 5.2 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/3.2.0/maven-jar-plugin-3.2.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/3.2.0/maven-jar-plugin-3.2.0.jar (31 kB at 31.0 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/34/maven-parent-34.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/34/maven-parent-34.pom (26 kB at 26.0 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/34/maven-parent-34.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/34/maven-parent-34.pom (26 kB at 26.0 MB/s)
[INFO] --- clean:3.2.0:clean (default-clean) @ MyProject ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.4.0/pom
```

```
C:\Users\USER>mvn git commit -m "updated"
[master (root-commit) 140bac0] updated
 6 files changed, 16 insertions(+), 0 deletions(-)
 create mode 1906044 pom.xml
 create mode 1906044 src/main/java/app.java
 create mode 1906044 target/hyProject-1.0.jar
 create mode 1906044 target/maven-archiver/pom.properties
 create mode 1906044 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
 create mode 1906044 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
 create mode 1906044 target/maven-status/maven-compiler-plugin/compile/default-compile/outputFiles.lst

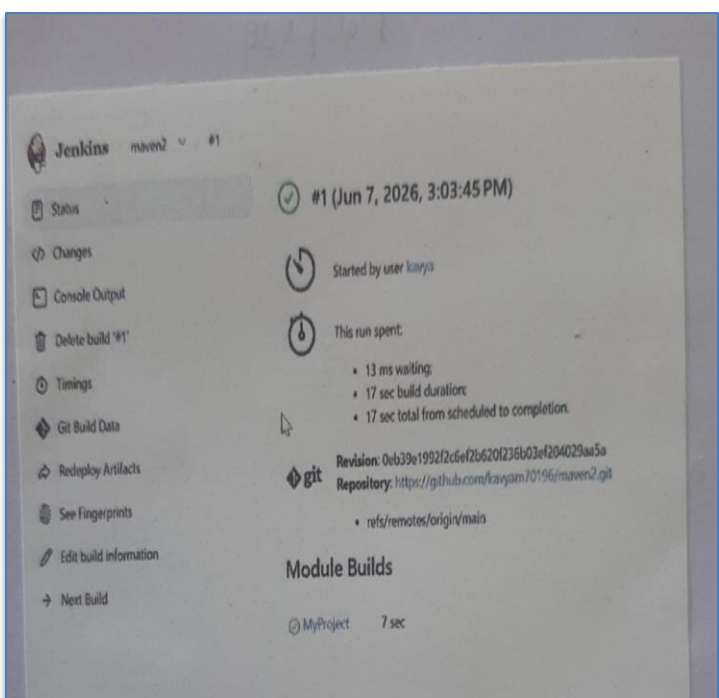
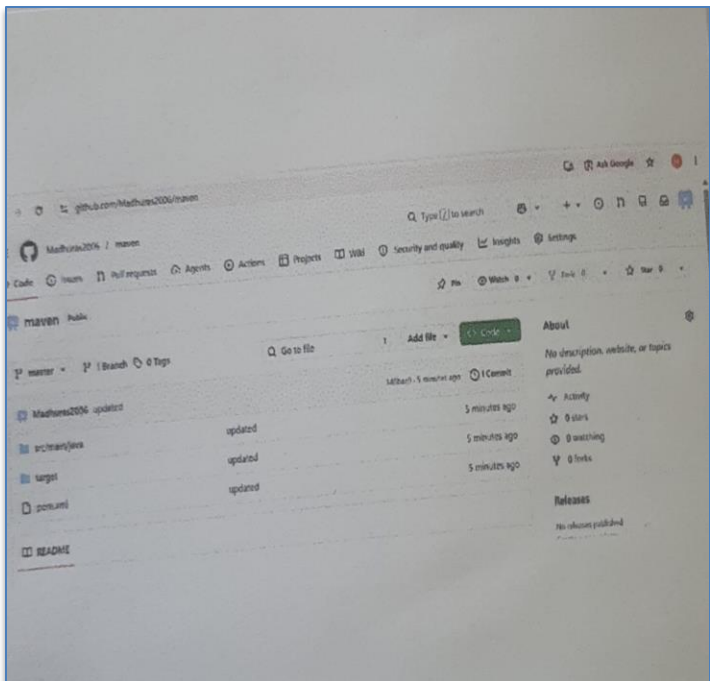
C:\Users\USER>mvn git remote add origin https://github.com/Madhuras2006/maven.git

C:\Users\USER>mvn git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/Madhuras2006/maven.git'

C:\Users\USER>mvn git remote -v
origin https://github.com/Madhuras2006/maven.git (fetch)
origin https://github.com/Madhuras2006/maven.git (push)

C:\Users\USER>mvn git push -u origin master
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 3 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (16/16), 1.72 KiB | 186.00 KiB/s, done.
Total 16 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Madhuras2006/maven.git
 * [new branch]      master -> master
 branch 'master' set up to track 'origin/master'.

C:\Users\USER>mvn
```



Program – 8

Apply Basic Commands for Docker and Integrate with Other Tools

AIM

To understand Docker architecture and basic Docker commands, create and containerize a Flask web application using Docker, and integrate Docker with a Python Flask application for container-based deployment.

OBJECTIVES

- Understand Docker architecture and containerization concepts.
- Create a simple Flask web application.
- Build a Docker image for the Flask application.
- Run the application inside a Docker container.
- Use Docker commands to manage images and containers.
- Verify application accessibility through a web browser.
- Demonstrate portability and environment consistency using Docker.

SOFTWARE REQUIREMENTS

- Python 3.8 or Above
- Docker Desktop / Docker Engine
- Flask Framework
- Command Prompt / Terminal
- Web Browser
- Internet Connection

PROCEDURE

Step 1

Install Python and Docker on the system.

Step 2

Create a Flask application using Python.

Step 3

Create the required source files:

- app.py
- main.py
- requirements.txt
- Dockerfile

Step 4

Define Flask dependencies in requirements.txt.

Step 5

Create a Dockerfile to build the application image.

Step 6

Build the Docker image using Docker commands.

Step 7

Run the Docker container.

Step 8

Access the application through a web browser.

Step 9

Verify the running container using Docker commands.

Step 10

Stop and remove the container.

Step 11

Verify Docker image and container management operations.

SOURCE CODE

File 1: app.py

```
from flask import Flask
```

```
app = Flask(__name__)
```

Explanation

- Imports Flask framework.
- Creates a Flask application object.

File 2: main.py

```
from app import app
```

```
@app.route('/')
```

```
def home():
```

```
    return 'Hello from Docker! Flask app is running inside a container.'
```

```
@app.route('/health')
```

```
def health():
```

```
    return 'OK', 200
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Explanation

- Imports Flask application object.
- Defines Home Route.
- Defines Health Check Route.
- Runs application on port 5000.

File 3: requirements.txt

```
flask==2.3.2
```

Explanation

Specifies Flask dependency required for application execution.

File 4: Dockerfile

```
FROM python:3.9-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 5000
```

CMD ["python", "main.py"]

Explanation

- FROM → Uses official Python Docker image.
- WORKDIR → Sets working directory.
- COPY → Copies project files.
- RUN → Installs dependencies.
- EXPOSE → Opens port 5000.
- CMD → Executes Flask application.

COMMANDS, EXPLANATION AND OUTPUT

Step 1: Create Project Directory

Command

MIT@DESKTOP-8N6978G MINGW64 ~

\$ mkdir FlaskDockerDemo

Command Explanation

Creates a new project directory.

Output

Directory Created Successfully

Step 2: Navigate to Project Folder

Command

MIT@DESKTOP-8N6978G MINGW64 ~

\$ cd FlaskDockerDemo

Command Explanation

Moves into the project directory.

Output

```
MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo
$
```

Step 3: Verify Docker Installation**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo
$ docker --version
```

Command Explanation

Displays Docker version information.

Output

```
Docker version 27.0.3, build xxxxx
```

Step 4: Build Docker Image**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo
$ docker build -t flask-app .
```

Command Explanation

- `docker build` → Builds Docker image.
- `-t flask-app` → Assigns image name.
- `.` → Uses current directory as build context.

Output

Sending build context to Docker daemon

Step 1/6 : FROM python:3.9-slim

---> 7d5d4f

Step 2/6 : WORKDIR /app

---> Running in 56c2f

Successfully built 8f7d56ab

Successfully tagged flask-app:latest

Step 5: Verify Docker Images

Command

MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo

\$ docker images

Command Explanation

Displays all Docker images stored locally.

Output

REPOSITORY	TAG	IMAGE ID	SIZE
------------	-----	----------	------

flask-app	latest	8f7d56ab	150MB
-----------	--------	----------	-------

python	3.9-slim	7d5d4f21	125MB
--------	----------	----------	-------

Step 6: Run Docker Container

Command

MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo

```
$ docker run -p 5000:5000 flask-app
```

Command Explanation

- `docker run` → Starts container.
- `-p 5000:5000` → Maps container port to host port.
- `flask-app` → Docker image name.

Output

* Running on all addresses (0.0.0.0)

* Running on http://127.0.0.1:5000

* Running on http://172.17.0.2:5000

Step 7: Access Application

Browser URL

`http://localhost:5000`

Output

Hello from Docker! Flask app is running inside a container.

Step 8: Verify Health Endpoint

Browser URL

`http://localhost:5000/health`

Output

OK

Step 9: List Running Containers**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo
```

```
$ docker ps
```

Command Explanation

Displays active running containers.

Output

```
CONTAINER ID   IMAGE      STATUS
```

```
ab12cd34ef56   flask-app   Up 2 minutes
```

Step 10: Inspect Container Details**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo
```

```
$ docker inspect ab12cd34ef56
```

Command Explanation

Displays detailed configuration information about a container.

Output

```
[
  {
    "Id": "ab12cd34ef56",
    "State": {
      "Running": true
    }
  }
]
```

]

Step 11: Stop Running Container

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo
```

```
$ docker stop ab12cd34ef56
```

Command Explanation

Stops the running container.

Output

```
ab12cd34ef56
```

Step 12: Remove Container

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo
```

```
$ docker rm ab12cd34ef56
```

Command Explanation

Deletes the stopped container.

Output

```
ab12cd34ef56
```

Step 13: Verify Available Images

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~/FlaskDockerDemo
```

```
$ docker images
```

Command Explanation

Lists all Docker images present on the system.

Output

REPOSITORY	TAG	IMAGE ID
------------	-----	----------

flask-app	latest	8f7d56ab
-----------	--------	----------

OUTPUT

Docker Image Created Successfully.

Container Started Successfully.

Application Accessible at:

<http://localhost:5000>

Response:

Hello from Docker! Flask app is running inside a container.

Health Check:

OK

Container Listed Successfully Using docker ps.

Container Stopped and Removed Successfully.

Program – 9**Title****Create and Apply Backlogs, Sprints, Tasks for a Project and Retrieve Data Using J-Query (Jira Query)****AIM**

To create and manage Product Backlogs, Sprints, and Tasks in Jira, and retrieve project information using Jira Query Language (JQL).

SOFTWARE REQUIREMENTS

- Jira Software
- Web Browser
- Internet Connection
- Jira Project Workspace
- User Account with Project Access

OBJECTIVES

- Create a Scrum Project in Jira.
- Create Product Backlog items.
- Create and manage Sprints.
- Create and assign Tasks.
- Track task progress.
- Retrieve project data using JQL queries.
- Monitor project activities through Jira dashboards.

PROCEDURE

Step 1

Login to Jira Software using valid credentials.

Step 2

Create a new Scrum Project.

Step 3

Create Product Backlog items such as User Stories, Tasks, and Bugs.

Step 4

Create a Sprint and add backlog items to the Sprint.

Step 5

Assign tasks to team members.

Step 6

Start the Sprint.

Step 7

Update task status during project execution.

Step 8

Use Jira Query Language (JQL) to retrieve project information.

Step 9

Generate reports and monitor Sprint progress.

Step 10

Review Sprint completion and close the Sprint.

COMMANDS, EXPLANATION AND OUTPUT**Step 1: Login to Jira****URL**

<https://your-domain.atlassian.net>

Explanation

Open Jira Software and login using authorized credentials.

Output

Jira Dashboard Opened Successfully

Step 2: Create Scrum Project

Steps

Projects → Create Project → Scrum → Create

Explanation

Creates a Scrum-based project for Agile development.

Example

Project Name : DevOps_Project

Project Key : DEV

Output

Project Created Successfully

Step 3: Create Product Backlog

Steps

Backlog → Create Issue

Example Backlog Items

Issue Key Issue Type Summary

DEV-1	Story	User Login Module
-------	-------	-------------------

DEV-2	Story	Student Dashboard
-------	-------	-------------------

Issue Key Issue Type Summary

DEV-3 Task Database Configuration

DEV-4 Bug Login Validation Error

Explanation

Backlog contains all pending project requirements.

Output

Backlog Items Added Successfully

Step 4: Create Sprint**Steps**

Backlog → Create Sprint

Example

Sprint Name : Sprint-1

Duration : 2 Weeks

Explanation

Sprint is a time-boxed iteration used to complete selected backlog items.

Output

Sprint Created Successfully

Step 5: Add Issues to Sprint**Steps**

Drag Backlog Items → Sprint-1

Explanation

Moves selected backlog items into the Sprint.

Output

DEPT. OFCSE-IC, MITM.

Issues Added to Sprint Successfully

Step 6: Start Sprint

Steps

Sprint-1 → Start Sprint

Example

Sprint Duration : 14 Days

Explanation

Begins Sprint execution for the selected tasks.

Output

Sprint Started Successfully

Step 7: Create and Assign Tasks

Steps

Issue → Assign

Example

Task	Assignee
User Login Module	Student A
Database Setup	Student B
Dashboard Design	Student C

Explanation

Assigns project work to team members.

Output

Tasks Assigned Successfully

Step 8: Update Task Status**Steps**

To Do → In Progress → Done

Explanation

Tracks progress of project activities.

Output

Task Status Updated Successfully

JQL COMMANDS**Command 1: View All Issues****JQL Command**

project = DEV

Explanation

Displays all issues belonging to the DEV project.

Output

DEV-1

DEV-2

DEV-3

DEV-4

Command 2: View All Open Issues**JQL Command**

project = DEV AND status = "To Do"

Explanation

Displays all pending tasks.

Output

DEV-2

DEV-3

Command 3: View Completed Tasks**JQL Command**

project = DEV AND status = Done

Explanation

Displays completed issues.

Output

DEV-1

Command 4: View Issues Assigned to User**JQL Command**

assignee = currentUser()

Explanation

Displays issues assigned to the currently logged-in user.

Output

DEV-2

DEV-3

Command 5: View Sprint Issues**JQL Command**

```
Sprint = "Sprint-1"
```

Explanation

Displays all issues assigned to Sprint-1.

Output

DEV-1

DEV-2

DEV-3

Command 6: View High Priority Issues**JQL Command**

```
priority = High
```

Explanation

Displays all high-priority issues.

Output

DEV-4

Command 7: View Bugs**JQL Command**

```
issuetype = Bug
```

Explanation

Displays all bug-related issues.

Output

DEV-4

Command 8: View Recently Created Issues

JQL Command

created >= -7d

Explanation

Displays issues created within the last seven days.

Output

DEV-2

DEV-3

DEV-4

Step 9: Generate Sprint Report**Navigation**

Reports → Sprint Report

Explanation

Displays Sprint progress and completion statistics.

Output

Completed Issues : 3

Incomplete Issues : 1

Sprint Progress : 75%

Step 10: Complete Sprint**Steps**

Sprint-1 → Complete Sprint

Explanation

Closes the Sprint and moves unfinished work to the backlog or next Sprint.

Output

Sprint Completed Successfully

OUTPUT

Scrum Project Created Successfully.

Product Backlog Created.

Sprint-1 Created and Started.

Tasks Assigned to Team Members.

Task Status Updated Successfully.

JQL Queries Executed Successfully.

Sprint Report Generated.

Sprint Completed Successfully.

Program – 10**Title****Create a Framework and Integrate Different DevOps Tools for an Application****AIM**

To create a DevOps framework by integrating different DevOps tools such as Git, GitHub, Jenkins, Maven, Docker, and Jira for automating software development, build, testing, deployment, and project management activities.

OBJECTIVES

- Understand the DevOps lifecycle.
- Create a source code repository using Git and GitHub.
- Automate build and testing using Jenkins and Maven.
- Containerize the application using Docker.
- Manage project activities using Jira.
- Implement Continuous Integration and Continuous Deployment (CI/CD).
- Integrate multiple DevOps tools into a unified framework.

SOFTWARE REQUIREMENTS

- Git
- GitHub Account
- Jenkins
- Apache Maven
- Docker

- Jira Software
- Java JDK
- Web Browser
- Internet Connection

DEVOPS FRAMEWORK

Developer

|

v

Git Local Repository

|

v

GitHub Remote Repository

|

v

Jenkins Pipeline

|

+----> Maven Build & Test

|

+----> Docker Image Creation

|

v

Deployment

|

v

Monitoring & Project Tracking (Jira)

PROCEDURE

Step 1

Create a local Git repository.

Step 2

Push source code to GitHub repository.

Step 3

Create project tasks and backlog items in Jira.

Step 4

Install and configure Jenkins.

Step 5

Configure Maven in Jenkins.

Step 6

Create a Jenkins Pipeline for CI/CD.

Step 7

Build and test the application using Maven.

Step 8

Create a Docker image for the application.

Step 9

Deploy the Docker container.

Step 10

Monitor project activities using Jira.

COMMANDS, EXPLANATION AND OUTPUT

Step 1: Create Local Repository**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ mkdir DevOpsFramework
```

Command Explanation

Creates a project directory.

Output

Directory Created Successfully

Command

```
MIT@DESKTOP-8N6978G MINGW64 ~
```

```
$ cd DevOpsFramework
```

Command Explanation

Moves into the project folder.

Output

```
MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework
```

```
$
```

Step 2: Initialize Git Repository**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework
```

```
$ git init
```

Command Explanation

Creates an empty Git repository.

Output

Initialized empty Git repository in DevOpsFramework/.git/

Step 3: Add Project Files**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework
```

```
$ git add .
```

Command Explanation

Adds all project files to the staging area.

Output

Files Added Successfully

Step 4: Commit Changes**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework
```

```
$ git commit -m "Initial Commit"
```

Command Explanation

Creates the first snapshot of the project.

Output

```
[master] Initial Commit
```

Step 5: Connect GitHub Repository**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework
```

```
$ git remote add origin https://github.com/student/DevOpsFramework.git
```

Command Explanation

Links the local repository with GitHub.

Output

Remote Repository Added Successfully

Step 6: Push Source Code**Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework
```

```
$ git push -u origin main
```

Command Explanation

Uploads source code to GitHub.

Output

```
Enumerating objects: 10
```

```
Writing objects: 100%
```

```
main -> main
```

Jira Integration**Step 7: Create Project in Jira****Navigation**

Projects → Create Project → Scrum Project

Example

Project Name : DevOps_Project

Project Key : DEV

Output

Project Created Successfully

Step 8: Create Backlogs and Sprints

Navigation

Backlog → Create Issue

Example Backlogs

DEV-1 : User Authentication

DEV-2 : Build Automation

DEV-3 : Docker Deployment

Output

Backlog and Sprint Created Successfully

Jenkins Integration

Step 9: Create Jenkins Pipeline Job

Navigation

Dashboard → New Item → Pipeline

Output

Pipeline Job Created Successfully

Step 10: Configure Maven Tool

Navigation

Manage Jenkins → Tools → Maven

Configuration

Name : M3

Output

Maven Configured Successfully

Step 11: Jenkins Pipeline Script

Pipeline Code

```
pipeline {  
    agent any  
  
    tools {  
        maven 'M3'  
    }  
  
    stages {  
  
        stage('Checkout') {  
            steps {  
                git branch: 'main',  
                url: 'https://github.com/student/DevOpsFramework.git'  
            }  
        }  
  
        stage('Build') {  
            steps {  
                sh 'mvn clean compile'  
            }  
        }  
    }  
}
```

```
}

stage('Test') {
    steps {
        sh 'mvn test'
    }
}

stage('Package') {
    steps {
        sh 'mvn package'
    }
}
}
```

Maven Lifecycle Commands

Build Project

Command

mvn clean compile

Explanation

Compiles source code and removes old build files.

Output

BUILD SUCCESS

Execute Test Cases**Command**

```
mvn test
```

Explanation

Runs all project test cases.

Output

Tests Passed Successfully

Package Application**Command**

```
mvn package
```

Explanation

Creates deployable JAR/WAR file.

Output

target/application.jar

Docker Integration**Step 12: Build Docker Image****Command**

```
MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework
```

```
$ docker build -t devops-app .
```

Command Explanation

Builds Docker image from project files.

Output

Successfully built 7ab9cd56

Successfully tagged devops-app:latest

Step 13: Verify Docker Images**Command**

MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework

\$ docker images

Output

REPOSITORY	TAG	IMAGE ID
------------	-----	----------

devops-app	latest	7ab9cd56
------------	--------	----------

Step 14: Run Docker Container**Command**

MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework

\$ docker run -d -p 8080:8080 devops-app

Command Explanation

Runs the application inside a Docker container.

Output

a7c3f2d8b1e6

Step 15: Verify Running Containers**Command**

MIT@DESKTOP-8N6978G MINGW64 ~/DevOpsFramework

```
$ docker ps
```

Output

CONTAINER ID	IMAGE	STATUS
--------------	-------	--------

a7c3f2d8b1e6	devops-app	Up 2 minutes
--------------	------------	--------------

Continuous Integration Workflow

Developer Commit

```
|
```

```
v
```

Git Repository

```
|
```

```
v
```

GitHub Push

```
|
```

```
v
```

Webhook Trigger

```
|
```

```
v
```

Jenkins Pipeline

```
|
```

```
+----> Maven Build
```

```
|
```

```
+----> Maven Test
```

```
|
```

+----> Docker Build

|

v

Deployment

|

v

Jira Monitoring

OUTPUT

Git Repository Created Successfully.

Source Code Uploaded to GitHub.

Jira Project Created.

Backlogs and Sprints Configured.

Jenkins Pipeline Created.

Maven Build Completed Successfully.

Test Cases Executed Successfully.

Docker Image Built Successfully.

Application Deployed Using Docker Container.

Continuous Integration Workflow Executed Successfully.